

URL Shortener Web Application Report

Introduction

The URL Shortener Web Application is a web-based application developed using Python Flask framework, SQLAlchemy ORM, SQLite database, HTML, CSS, and Bootstrap. The main objective of this project is to convert long URLs into shorter and easy-to-share links. In today's digital communication, users frequently share website links through emails, social media platforms, messaging applications, and other online platforms. However, lengthy URLs are often difficult to manage, remember, and share efficiently. A URL shortener solves this problem by generating compact URLs that redirect users to the original website.

This project was developed as part of a practical web development assignment to understand frontend and backend integration using Flask. The application not only shortens URLs but also stores them in a database so users can view previously shortened URLs through a History page. The project provided hands-on experience in building a full-stack web application and implementing database operations using SQLAlchemy ORM.

Project Objective

The primary objective of this project is to develop a fully functional URL Shortener Web Application using Flask and SQLite database. The application should allow users to enter a long URL, generate a unique shortened URL, and redirect users back to the original website when the shortened URL is accessed.

Another important objective of the project is to store the original URLs and shortened URLs in a database so that users can access previously shortened URLs through a separate History page. The project also aims to validate user-entered URLs before storing them in the database to ensure that only valid URLs are processed.

In addition, the project was designed to improve understanding of frontend technologies such as HTML, CSS, and Bootstrap, as well as backend technologies such as Flask routing, form handling, database integration, and URL redirection.

Problem Statement

Long URLs are often difficult to copy, paste, remember, and share. In many situations, users need a shorter and cleaner link that can be easily distributed through different communication platforms. Sharing lengthy URLs may also look unprofessional and inconvenient for users.

The purpose of this project is to create a URL Shortener Web Application that simplifies the process of sharing URLs by generating shorter versions of long links. The application also

provides functionality to save and manage shortened URLs in a database so that users can revisit them later through a History page.

Technologies Used

Several technologies were used to develop this project. Python was used as the primary programming language because of its simplicity and strong support for web application development. Flask was used as the backend web framework to handle routing, request processing, and communication between the frontend and backend.

SQLAlchemy ORM was used for database operations. It simplified database interactions by allowing database tables to be managed using Python classes instead of writing raw SQL queries. SQLite was selected as the database because it is lightweight, easy to configure, and suitable for beginner-level projects.

The frontend of the application was developed using HTML, CSS, and Bootstrap. HTML was used to create the structure of the webpages, CSS was used for styling, and Bootstrap was used to make the application responsive and visually attractive. JavaScript was also used to implement the “Copy URL” functionality.

The validators library was used to validate user-entered URLs before storing them in the database.

System Architecture

The application follows a client-server architecture where the frontend interacts with the backend server to process user requests. The frontend contains webpages that allow users to enter URLs and view results. The backend developed using Flask processes these requests, interacts with the database, and returns appropriate responses.

The frontend consists of two webpages:

1. Home Page.
2. History Page.

The Home Page allows users to enter a URL and generate a shortened version. The History Page displays all previously shortened URLs stored in the database.

The backend handles URL validation, short code generation, database insertion, and URL redirection. SQLite database stores all URL-related information including original URLs and shortened codes.

Frontend Development

The frontend of the application was developed using HTML, CSS, and Bootstrap. Bootstrap was used to make the user interface responsive and visually appealing. The Home Page

contains an input field where users can enter the URL they want to shorten. After clicking the “Shorten URL” button, the shortened URL is displayed in a text field.

A “Copy URL” button was implemented using JavaScript to allow users to copy the shortened URL directly to the clipboard. This improves usability and provides a better user experience.

The History Page displays all stored URLs in a tabular format. The table contains two columns: Original URL and Shortened URL. Users can click on the shortened URL to redirect to the original website.

CSS styling was added to improve the appearance of forms, buttons, tables, and overall page layout.

Backend Development

The backend was developed using Flask framework in Python. Flask routes were created to handle different functionalities of the application such as displaying webpages, processing form data, shortening URLs, storing data in the database, and redirecting users to original URLs.

The main routes implemented in the project are:

- / → Home Page
- /history → History Page
- /<short_code> → Redirect Route

The Home route handles URL input and generates shortened URLs. The History route retrieves stored URLs from the database and displays them on the webpage. The Redirect route identifies the original URL associated with the shortened code and redirects users to the destination website.

Flask template rendering was used to dynamically pass data from the backend to HTML pages.

Database Integration

SQLite database was used to store URL data permanently. SQLAlchemy ORM was used to define database models and interact with the database.

A database table named URLMap was created with the following fields:

- ID.
- Original URL.
- Short Code.

The Original URL field stores the long URL entered by the user. The Short Code field stores the randomly generated shortened code associated with the URL.

Whenever a user shortens a URL, the data is inserted into the database. The History page retrieves all stored records from the database and displays them to the user.

The database file `urls.db` is automatically created when the Flask application runs successfully for the first time.

URL Shortening Logic

The URL shortening functionality was implemented using Python's `random` and `string` libraries. A random alphanumeric code containing uppercase letters, lowercase letters, and digits is generated for each URL.

For example:

Original URL:

`https://www.google.com`

Generated Short Code:

`aB12Cd`

Shortened URL:

`http://127.0.0.1:5000/aB12Cd`

The shortened URL is stored in the database along with the original URL. When the shortened URL is accessed, the application searches for the corresponding original URL in the database and redirects the user accordingly.

URL Validation

URL validation was implemented using the Python `validators` library. Before shortening any URL, the application checks whether the entered URL is valid.

If the entered URL is invalid, an error message is displayed to the user and the URL is not stored in the database. This validation improves data integrity and prevents incorrect URLs from being processed.

Example validation:

```
validators.url(original_url)
```

This feature ensures that only properly formatted URLs are accepted by the application.

Project Workflow

The workflow of the application begins when the user opens the Home Page. The user enters a long URL into the input field and clicks the "Shorten URL" button.

The backend receives the entered URL and first validates it using the `validators` library. If the URL is valid, the application generates a unique short code using random alphanumeric characters.

The original URL and generated short code are then stored in the SQLite database using SQLAlchemy ORM. The shortened URL is displayed on the screen along with a Copy button. The user can copy the shortened URL and share it with others. When someone accesses the shortened URL, Flask identifies the corresponding original URL from the database and redirects the user to the original website.

Users can also navigate to the History Page to view all previously shortened URLs stored in the database.

Challenges Faced During Development

Several challenges were encountered during the development process. One of the major challenges was understanding how Flask routes interact with HTML templates and form data. Managing user input and dynamically displaying results required careful implementation.

Another challenge was integrating SQLite database using SQLAlchemy ORM. Understanding how to define database models, create tables, and perform database operations required additional learning and experimentation.

Implementing URL redirection was also challenging because the application needed to dynamically identify original URLs based on shortened codes.

Another important challenge was validating URLs before storing them in the database. This issue was solved using the validators library.

These challenges significantly improved debugging skills and understanding of full-stack web application development.

Testing and Results

The application was tested using multiple valid and invalid URLs to ensure proper functionality. Valid URLs such as Google, YouTube, and GitHub links were successfully shortened and redirected correctly.

Example test:

Input URL:

`https://www.youtube.com`

Generated Short URL:

`http://127.0.0.1:5000/Xy12Ab`

Result:

The shortened URL successfully redirected users to the original YouTube website.

The History page was also tested and correctly displayed all previously shortened URLs stored in the database.

Invalid URLs were successfully rejected with appropriate error messages.

Overall, the application performed successfully across all major functionalities including URL shortening, validation, database storage, history management, and redirection.

Learning Outcomes

This project provided valuable practical exposure to Flask-based web development and full-stack application development. Through this project, several important concepts were learned including Flask routing, form handling, template rendering, SQLAlchemy ORM, database integration, URL validation, and URL redirection.

The project also improved frontend development skills through the use of HTML, CSS, and Bootstrap. Understanding how frontend and backend systems communicate helped strengthen overall web development knowledge.

Working with SQLite database improved understanding of database operations, data storage, and retrieval processes. Most importantly, the project improved debugging, logical thinking, and problem-solving abilities while working on a real-world web application.

Future Enhancements

Several advanced features can be added to improve the application further. User authentication functionality can be added so users can create accounts and manage their own URLs separately.

Additional enhancements may include:

- Custom short URLs.
- Password protection for URLs.
- QR code generation.
- Click analytics.
- URL expiration dates.
- Cloud deployment.
- REST API integration.
- Dark mode interface.

These features would improve the usability, scalability, and security of the application.

Conclusion

The URL Shortener Web Application was successfully developed using Flask, SQLAlchemy, SQLite, HTML, CSS, Bootstrap, and JavaScript. The application successfully shortens long URLs, validates user input, stores data in a database, displays URL history, and redirects users to original websites.

This project provided practical experience in frontend development, backend development, database integration, and full-stack web application architecture. It also improved understanding of Flask framework, SQLAlchemy ORM, and web application workflows.

Overall, the project was a highly valuable learning experience that strengthened both technical and problem-solving skills in web development.